CSC11006

Introduction to Cloud Computing Services

PROJECT 2

Building a Serverless Web Application with Zero-Cost & Secure Architecture

I. General Information

Field	Details
Project ID	PROJECT2
Duration	3 weeks
Team size	Group of 3 students
Submission deadline	End of Week 3 (see course schedule)

II. Learning Outcomes

This project addresses the following course learning outcomes: G2.1, G2.2, G2.3, G2.4, G3.1, G5.2, G5.3

- Design cloud systems using managed serverless services
- Implement security with Least Privilege principles and network isolation
- Configure monitoring, alerting, and cost management
- Build and deploy a complete web application on a cloud platform
- Explain system architecture, request flow, and auto-scaling mechanisms

III. Project Description

Students will design and deploy a Task Manager web application using a modern serverless architecture on AWS. The system must demonstrate auto-scaling, defence-in-depth security, and database connectivity entirely over the AWS private network — never traversing the public internet.

Core Objectives

- 100% serverless architecture — no virtual machines (EC2) allowed
- Automatic scaling based on real traffic
- High Availability (HA) by design — no manual failover configuration needed
- Network isolation: Lambda must connect to DynamoDB via VPC Endpoint, not the internet
- Frontend security: S3 bucket must be private, accessible only through CloudFront + OAC
- Strict Zero-Cost / Free Tier constraint
- Full observability and cost control

IV. Technical Requirements

1. Application Design — Frontend

1.1 Technology stack and features

- Tech stack: HTML, CSS, JavaScript (no frontend framework required)
- Required features:
  - CRUD operations: create, read, update, delete tasks
  - Responsive task list interface
  - Filter tasks by due date and priority

1.2 Frontend Hosting — Security Requirements (CRITICAL UPDATE)

Prohibited — Do NOT do this
The S3 bucket must NOT be configured in public mode.
Static website hosting on S3 must NOT be enabled.
Files must NOT be accessible via a direct S3 URL such as: https://bucket.s3.amazonaws.com/...

Required — Correct approach
The S3 bucket must be PRIVATE — all four Block Public Access options must be enabled.
CloudFront must be the sole CDN serving the frontend to users.
Origin Access Control (OAC) must be configured so CloudFront can read from S3.
The S3 Bucket Policy must allow only the CloudFront Service Principal to read objects.

1.3 Frontend Security Evidence — MANDATORY SUBMISSION

Teams must submit all four items listed below. Missing any single item will result in point deductions under the Security category.

ID	Evidence item	How to collect	Accepted format
SE-1	S3 Block Public Access enabled	Screenshot of the S3 bucket Permissions tab showing all four checkboxes enabled	Screenshot
SE-2	Direct S3 access rejected	Open a browser or run curl against https://<bucket>.s3.amazonaws.com/index.html and capture the 403 Forbidden / AccessDenied response	Screenshot or curl output
SE-3	CloudFront access succeeds	Open the CloudFront URL (https://<id>.cloudfront.net) and capture the working webpage with HTTP status 200	Screenshot
SE-4	OAC attached to CloudFront	Screenshot of the CloudFront distribution Settings page clearly showing 'Origin access: Origin access control settings' and the OAC name	Screenshot

2. Backend API

2.1 Language and API endpoints

- Language: Node.js 20.x or Python 3.12
- Required endpoints:

Method	Path	Description
GET	/tasks	Retrieve all tasks
POST	/tasks	Create a new task
PUT	/tasks/:id	Update a task by ID
DELETE	/tasks/:id	Delete a task by ID

2.2 Compute architecture — mandatory requirements

- Serverless compute: AWS Lambda
- Event-driven: Lambda triggered by API Gateway
- Stateless: each request is fully independent; no state stored inside Lambda
- Auto-scaling: Lambda scales automatically — no additional configuration required

3. Networking — Lambda to DynamoDB Connectivity (PRECISE REQUIREMENT)

Technical explanation
The core networking challenge: Lambda needs to read/write DynamoDB. There are two approaches: Approach 1 (WRONG): Lambda calls DynamoDB over the public internet — requires NAT Gateway — costs ~$32/month.
Approach 2 (CORRECT): Lambda calls DynamoDB via a VPC Gateway Endpoint — traffic stays on the AWS private backbone — costs $0. This project mandates Approach 2.

3.1 Required VPC configuration

Students must create a Custom VPC and deploy Lambda functions inside that VPC:

Component	Configuration	Purpose
Custom VPC	CIDR: 10.0.0.0/16	Private virtual network to isolate compute
Private Subnet AZ-1	Example: 10.0.1.0/24 in ap-southeast-1a	Place Lambda functions in this subnet
Private Subnet AZ-2	Example: 10.0.2.0/24 in ap-southeast-1b	Ensures High Availability across two AZs
VPC Gateway Endpoint	Create a DynamoDB endpoint and associate it with the Route Table	Allows Lambda to reach DynamoDB privately
Lambda Security Group	Outbound: allow port 443 to the DynamoDB Prefix List only	Prevents Lambda from reaching the public internet

Component	Configuration	Purpose
Route Table entry	Destination: pl-xxxxxx (DynamoDB Prefix List) -> vpce-xxxxxxxx	Directs DynamoDB traffic through the Endpoint

3.2 Attaching Lambda to the VPC — mandatory step

When creating each Lambda function, you must configure VpcConfig. Without this field, Lambda runs outside your Custom VPC and will not use the Endpoint you created.

⚠ **VpcConfig — MANDATORY for every Lambda function**

SubnetIds: list both private subnet IDs (AZ-1 and AZ-2).
SecurityGroupIds: attach the Lambda Security Group with outbound port 443 to DynamoDB Prefix List only.
Effect: Lambda attaches an ENI to the subnet — this is what routes traffic through the VPC Gateway Endpoint.
Without VpcConfig, Lambda runs OUTSIDE the VPC and cannot use the DynamoDB Endpoint you created.
The Architecture Diagram must explicitly show Lambda attached to the VPC via VpcConfig, not just pointing into the VPC box.

Prohibited — Do NOT do this
Creating or using a NAT Gateway is strictly prohibited.
NAT Gateway is billed at $0.045/hour = ~$32/month even with zero traffic. Violating this rule results in zero marks for both Networking and Cost Efficiency.
Instructors will verify compliance by inspecting the VPC console — any NAT Gateway found = 0 points.

3.3 Networking evidence — mandatory submission

ID	Evidence item	How to collect	Accepted format
NE-1	VPC Endpoint created for DynamoDB	Screenshot of VPC > Endpoints showing Service Name: com.amazonaws.<region>.dynamodb and Status: Available	Screenshot
NE-2	Lambda has VpcConfig	Screenshot of the Lambda function's Configuration > VPC tab showing Subnets and Security Group attached	Screenshot
NE-3	Route table has Endpoint route	Screenshot of the Private Subnet Route Table showing a row: Destination = pl-xxxxxx (DynamoDB Prefix List), Target = vpce-xxxxxxxx	Screenshot
NE-4	No NAT Gateway exists	Screenshot of VPC > NAT Gateways — the list must be empty or all entries must show Status: Deleted	Screenshot
NE-5	Lambda successfully calls DynamoDB	CloudWatch log of a Lambda invocation showing StatusCode 200 on the DynamoDB call, with no NetworkingError or UnauthorizedAccess error	Screenshot of log

4. Database — Amazon DynamoDB

- Type: NoSQL — Amazon DynamoDB
- High Availability is built in by default (multi-AZ replication, no configuration needed)
- Free Tier allowance: 25 GB storage, 25 RCU / 25 WCU per month (no expiry)

4.1 Table schema

Attribute	Type	Role	Description
taskId	String	Partition Key	UUID generated by Lambda on task creation
userId	String	GSI partition key	ID of the user who owns the task
title	String	Attribute	Task title
description	String	Attribute	Optional task details
priority	String	Attribute	Values: low / medium / high
dueDate	String	Attribute	ISO format: 2025-06-15
status	String	Attribute	Values: pending / done
createdAt	String	Attribute	ISO timestamp set at creation time

4.2 Global Secondary Index (GSI)

Create a GSI to enable efficient querying of all tasks belonging to a given user:

- GSI name: userId-index
- GSI partition key: userId
- Projection: ALL

Without this GSI, fetching a user's tasks requires a full table Scan — which is inefficient and will not satisfy the architecture design requirement.

5. Security — IAM Least Privilege

5.1 Permission requirements

Each Lambda function must have its own dedicated IAM Role. Sharing a single role across multiple Lambda functions is not permitted.

IAM Role	Lambda	Allowed actions (Allow)	Strictly prohibited
LambdaTaskRole	Task Service	dynamodb:GetItem, PutItem, UpdateItem, DeleteItem, Query, Scan on the TasksTable ARN only logs:CreateLogGroup, CreateLogStream, PutLogEvents ec2:CreateNetworkInterface, DescribeNetworkInterfaces, DeleteNetworkInterface (required for VPC attachment)	Wildcard (*) actions or resources Access to any table other than TasksTable
LambdaBaseRole	Other Lambda functions	logs:CreateLogGroup, CreateLogStream, PutLogEvents ec2:CreateNetworkInterface, DescribeNetworkInterfaces, DeleteNetworkInterface	Any DynamoDB access

Note

The ec2:CreateNetworkInterface permission is required when Lambda runs inside a VPC.

Without it, Lambda deployment fails with: 'Lambda was unable to configure VPC access'.

This is not a security concern — it is a technical requirement for Lambda to attach an ENI to the subnet.

5.2 IAM evidence — mandatory submission

ID	Evidence item	How to collect
IM-1	Two separate IAM Roles	Screenshot of IAM > Roles filtered by project name, showing two distinct roles
IM-2	Policy specifies exact resource ARN	Open the Inline Policy or JSON of each Role and capture the 'Resource' field — it must show the specific DynamoDB table ARN, not '*'
IM-3	Lambda attached to the correct Role	Screenshot of each Lambda function's Configuration > Permissions tab showing the correct Role name

6. API Gateway — Routing and Security

6.1 Required configuration

- API type: REST API (do not use HTTP API for this project)
- Deployment stage: prod
- HTTPS: enabled by default — API Gateway provisions the SSL certificate automatically

6.2 Rate limiting (throttling)

Parameter	Recommended value	Rationale
Rate (req/s)	100	Limits request throughput per user
Burst	50	Maximum simultaneous requests allowed
Lambda Reserved Concurrency	50	Prevents Lambda from scaling beyond the Free Tier

6.3 CORS — security requirement

- Access-Control-Allow-Origin: must be set to your CloudFront domain only
- Example: https://d1abc123.cloudfront.net
- Setting '*' is not permitted and will result in point deductions

7. Monitoring and Cost Management

7.1 CloudWatch Dashboard

Create a CloudWatch Dashboard named TaskManager-Dashboard with at least the following five widgets:

Widget	Metric	Namespace	Purpose
Invocations	Invocations	AWS/Lambda	Total request count
Duration	Duration (P50, P99)	AWS/Lambda	Lambda execution latency
Errors	Errors	AWS/Lambda	Function-level errors
Throttles	Throttles	AWS/Lambda	Requests dropped by concurrency cap
API Latency	Latency	AWS/ApiGateway	End-to-end response time
4xx / 5xx	4XXError, 5XXError	AWS/ApiGateway	Client and server error rates

7.2 CloudWatch Alarms — mandatory

Create at least two alarms:

Alarm name	Condition	Action
Lambda-Error-Alarm	Lambda Errors > 10 in 5 minutes	Send email via SNS
API-5xx-Alarm	API Gateway 5XXError > 5 in 5 minutes	Send email via SNS

7.3 Log evidence — mandatory

Teams must submit two CloudWatch log screenshots as part of the Deliverables:

Log	Content required	Log group location
Successful request	Log showing the REPORT line with StatusCode 200, Duration, Billed Duration, and Memory Used	/aws/lambda/<function-name>
Error request	Log showing an ERROR entry or stack trace triggered by sending invalid input (e.g. missing required field)	/aws/lambda/<function-name>

7.4 Cost control — mandatory

- Create an AWS Budget with a limit of $0.01 per month
- Configure alerts at 80% ($0.008) and 100% ($0.01) — both must send email notifications
- Set Lambda Reserved Concurrency = 50 to prevent accidental scaling beyond the Free Tier
- Apply API Gateway throttling as specified in Section 6.2

V. Deliverables

1. Web Application

- Frontend accessible via the CloudFront URL
- Backend API returning correct JSON responses
- All four CRUD endpoints functioning correctly
- Database integration verified — data persists in DynamoDB across requests

2. Architecture Diagram

A diagram drawn with any tool (draw.io, Lucidchart, hand-drawn, etc.) must clearly show ALL of the following components. The diagram is graded on completeness — missing any item results in deductions.

Layer	Required component
Edge	CloudFront + OAC + private S3
API	API Gateway → Lambda trigger
Compute (VPC)	Lambda with VpcConfig inside Private Subnets
Network	VPC Gateway Endpoint for DynamoDB
Database	DynamoDB
Security	IAM — two separate Roles
Monitoring	CloudWatch + SNS alarms
Cost	AWS Budgets

3. Security Evidence Package

Submit all evidence items listed in the table below. Files may be compiled as a PDF or a folder of PNG/JPG screenshots:

ID	Evidence item	Rubric category
SE-1	S3 Block Public Access — all four options enabled	Cloud Deployment
SE-2	Direct S3 access returns 403 Forbidden	Cloud Deployment
SE-3	CloudFront access returns 200 OK	Cloud Deployment
SE-4	OAC attached to CloudFront distribution	Cloud Deployment
NE-1	VPC Endpoint for DynamoDB — Status: Available	Architecture Understanding
NE-2	Lambda VpcConfig — Subnets and Security Group attached	Architecture Understanding
NE-3	Route Table — row pl-xxx -> vpce-xxx present	Architecture Understanding
NE-4	No NAT Gateway exists (empty list or all Deleted)	Cost Efficiency
NE-5	CloudWatch log: Lambda calls DynamoDB successfully	Monitoring
IM-1	Two separate IAM Roles with distinct names	Cloud Deployment
IM-2	Policy JSON with specific DynamoDB table ARN as Resource	Cloud Deployment
IM-3	Each Lambda attached to the correct Role	Cloud Deployment

4. Monitoring Dashboard

- Screenshot of the CloudWatch Dashboard showing at least 5 widgets with live data
- Screenshot of the 2 alarms created (status: OK or ALARM — both are acceptable)
- Screenshot of the AWS Budget configuration page

5. Cost Report

- Screenshot of AWS Cost Explorer or the Billing Dashboard
- Demonstrate total cost <= $0.01 throughout the entire project duration
- If any cost was incurred, provide an explanation and describe the corrective action taken

6. Documentation

The technical write-up must explain the following four concepts clearly. Instructors will ask about these directly during the project presentation:

Concept	What must be explained	Approximate weight
Request flow	Describe every step from when a user clicks 'Create task' to when the response is received: Browser → CloudFront (for static assets) or API GW → Lambda → VPC Endpoint → DynamoDB → response	~8%
Serverless auto-scaling	Explain how Lambda scales from 0 to N concurrent instances. Describe how API Gateway acts as a native load balancer. Explain why no Application Load Balancer (ALB) is needed	~6%
CDN and OAC	Explain how CloudFront caching works (cache HIT vs MISS). Define OAC and why it is required. Explain why the S3 bucket must remain private and not use static website hosting	~4%
VPC Endpoint	Explain why Lambda inside a Private Subnet needs a VPC Endpoint to reach DynamoDB. Describe the traffic path (AWS private backbone). Compare with NAT Gateway in terms of cost and security	~4%

7. Source Code

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js or Python — all four CRUD handlers included
- (Optional) CloudFormation or AWS SAM template
- README describing how to run locally and how to deploy to AWS

VI. Evaluation Criteria (Rubric)

Category	Weight	Requirements for full marks
Functionality	20%	All four CRUD operations work correctly. Frontend, backend, and DynamoDB are properly integrated. Priority and due date filters function as expected.
Cloud Deployment	25%	S3 private + CloudFront OAC verified (SE-1 to SE-4). Lambda inside VPC with Endpoint (NE-1 to NE-3). IAM Least Privilege with separate roles (IM-1 to IM-3). API Gateway CORS restricted to CloudFront domain only.
Architecture Understanding	20%	Student can explain the full request flow end-to-end. Student understands VPC Endpoint and why it replaces NAT Gateway. Student understands Lambda auto-scaling and API Gateway's load balancing role. Student understands CloudFront CDN caching and OAC.
Monitoring	10%	CloudWatch Dashboard with 5+ widgets showing live data. Two alarms created and configured. Successful and

Category	Weight	Requirements for full marks
		error log screenshots submitted (NE-5). SNS email notification configured.
Cost Efficiency	15%	No NAT Gateway present (NE-4). Lambda concurrency capped at 50. AWS Budget at $0.01 configured. Total project cost = $0 or negligibly close to $0.
Documentation	10%	Clear write-up explaining all four concepts. Architecture Diagram includes all required components. All 12 evidence items (SE/NE/IM) submitted in full.

Total: 100%. Missing any mandatory evidence item will result in point deductions in the corresponding rubric category, regardless of whether the system is functionally correct.

VII. Project Timeline

Week	Main tasks	Expected outcome by end of week
Week 1	Develop frontend and backend locally	Frontend runs locally (npm serve or Live Server). All four Lambda CRUD handlers written and tested locally using DynamoDB Local or a mock library.
Week 2	Deploy cloud infrastructure and connect the database	Custom VPC + 2 Private Subnets + VPC Endpoint created. Lambda deployed inside the VPC. DynamoDB table and GSI created. API Gateway configured with CORS and throttling. CloudFront + OAC + private S3 bucket live. All five networking evidence items NE-1 to NE-5 captured.
Week 3	Testing, monitoring setup, documentation, and submission	CloudWatch Dashboard + 2 Alarms + 2 log screenshots completed. AWS Budget configured. Cost Report ready. Full documentation written. All 12 evidence items (SE/NE/IM) compiled and submitted.

VIII. Important Notes

Common mistakes to avoid

Mistake	Description	Consequence
Public S3 bucket	Leaving the bucket in public mode or enabling Static Website Hosting on S3	Full deduction on SE-1 through SE-4 (Cloud Deployment)
Using a NAT Gateway	Creating a NAT Gateway to allow Lambda inside a VPC to reach the internet	Full deduction on NE-4 and Cost Efficiency. Incurs ~$32/month in charges.
Lambda outside the VPC	Creating Lambda without configuring VpcConfig — Lambda runs outside the Custom VPC	Full deduction on NE-1 through NE-3 even if the VPC Endpoint was correctly created

Mistake	Description	Consequence
Shared IAM Role	Using a single Role for multiple Lambda functions, or using AdministratorAccess / AmazonDynamoDBFullAccess with a wildcard resource	Full deduction on IM-1 through IM-3
CORS wildcard	Setting Access-Control-Allow-Origin: * instead of the specific CloudFront domain	Point deduction under Cloud Deployment (Security)
Missing evidence items	Not capturing screenshots according to the SE/NE/IM table	Point deductions per missing item, even if the system is functionally correct

Pre-submission checklist

1. S3 bucket: all four Block Public Access options = ON (SE-1)
2. Test direct S3 URL access — must return 403 Forbidden (SE-2)
3. Test CloudFront URL access — must return 200 OK (SE-3)
4. CloudFront distribution: Origin access = OAC (SE-4)
5. VPC > Endpoints: DynamoDB endpoint present, Status = Available (NE-1)
6. Lambda > Configuration > VPC: Subnets and Security Group shown (NE-2)
7. Private Subnet Route Table: row pl-xxx -> vpce-xxx present (NE-3)
8. VPC > NAT Gateways: list is empty or all entries show Status = Deleted (NE-4)
9. CloudWatch log: Lambda called DynamoDB successfully, no NetworkError (NE-5)
10. IAM: 2 separate Roles, each Policy uses a specific table ARN not '*' (IM-1, IM-2, IM-3)
11. CloudWatch Dashboard: 5+ widgets displaying real data
12. 2 CloudWatch Alarms created and visible
13. AWS Budget $0.01 configured and email subscription confirmed
14. Successful + error log screenshots captured from CloudWatch
15. Documentation covers all four concepts: request flow, scaling, CDN, VPC Endpoint
16. Architecture Diagram explicitly shows Lambda VpcConfig inside VPC boundary
17. Architecture Diagram includes IAM roles, CloudWatch, SNS, and AWS Budgets
18. Demo video link for backup scenario